

EXPERIMENT NO. 1 • BASIC ARITHMETIC OPERATIONS

Complete Solutions for Basic, Intermediate & Advanced Practice Problems

BASIC PRACTICE PROBLEMS (Q1 & Q2)**Q1: Add Numbers at 2060H & 2061H → Store at 2062H**

```
LDA 2060H ; Load 1st byte MOV B,A ; Cache in
reg B LDA 2061H ; Load 2nd byte ADD B ; A = A + B STA
2062H ; Store Sum HLT
```

Inst	A	B	CY	Memory Update
LDA 2060H	25H	--	0	Read 2060H (25H)
MOV B,A	25H	25H	0	Cached in B
LDA 2061H	32H	25H	0	Read 2061H (32H)
ADD B	57H	25H	0	25H + 32H = 57H
STA 2062H	57H	25H	0	2062H ← 57H

Q2: Subtract 2061H from 2060H → Store at 2062H

```
LDA 2060H ; Load Minuend MOV B,A LDA 2061H ;
Load Subtrahend MOV C,A MOV A,B ; Restore Minuend SUB C
; A = Minuend - Subtrahend STA 2062H ; Store Difference
HLT
```

Inst	A	B	C	Memory Update
LDA 2060H	48H	--	--	Read 2060H (48H)
LDA 2061H	15H	48H	--	Read 2061H (15H)
SUB C	33H	48H	15H	48H - 15H = 33H
STA 2062H	33H	48H	15H	2062H ← 33H

INTERMEDIATE PRACTICE PROBLEMS (Q1 & Q2)**Q1: Add Three Consecutive Numbers (2050H..2052H) → 2053H**

```
LXI H,2050H ; HL points to 1st byte MOV A,M ; A
= 1st byte INX H ADD M ; A = 1st + 2nd INX H ADD M ; A =
Sum + 3rd INX H MOV M,A ; Store at 2053H HLT
```

Step	HL	M	Accumulator (A)	Remarks
1	2050H	12H	12H	1st byte loaded
2	2051H	15H	12H + 15H = 27H	2nd byte added
3	2052H	20H	27H + 20H = 47H	3rd byte added
4	2053H	--	47H	2053H ← 47H

Q2: Subtract Immediate Value (08H) from 2050H → 2051H

```
LDA 2050H ; Load input byte SUI 08H ; Subtract
immediate 08H STA 2051H ; Store difference HLT
```

Inst	A	CY	Target Memory	Remarks
LDA 2050H	30H	0	--	2050H = 30H loaded
SUI 08H	28H	0	--	30H - 08H = 28H
STA 2051H	28H	0	2051H ← 28H	Result stored

ADVANCED PRACTICE PROBLEMS (Q1 & Q2)**Q1: Add Four Consecutive Numbers (2050H..2053H) → 2054H**

```
LXI H,2050H MVI C,04H ; Counter = 4 XRA A ; Sum
= 0 SUM4: ADD M INX H DCR C JNZ SUM4 STA 2054H HLT
```

Iter	Addr	Data	Sum (A)	Count (C)
1	2050H	10H	10H	03H
2	2051H	15H	25H	02H
3	2052H	20H	45H	01H
4	2053H	25H	6AH	00H (Done)
Store	2054H	--	2054H ← 6AH	--

Q2: Add 2 Bytes with Sum (2052H) & Carry (2053H) via Branch

```
LDA 2050H MOV B,A LDA 2051H ADD B STA 2052H ;
Store Sum JNC NO_CY MVI A,01H ; Carry = 01H JMP ST_CY
NO_CY: MVI A,00H ; Carry = 00H ST_CY: STA 2053H HLT
```

Input 1	Input 2	Sum (2052H)	CY Flag	Carry (2053H)
F0H	20H	10H	1	01H
15H	22H	37H	0	00H

EXPERIMENT NO. 2 • DECISION MAKING & COMPARISON PROGRAMS

Even/Odd, Positive/Negative, 3-Number Extrema, Array Matches, and Range Checks

BASIC PRACTICE PROBLEMS (Q1 & Q2)

Q1: Even / Odd Check (2050H) → 2051H (00H=Even, 01H=Odd)

```
LDA 2050H ANI 01H ; Test LSB (Bit 0) JZ IS_EVEN
MVI A,01H ; Odd indicator JMP STORE_RES IS_EVEN: MVI
A,00H ; Even indicator STORE_RES: STA 2051H HLT
```

Input (2050H)	Bit 0	Z Flag	Stored (2051H)	Remarks
34H (00110100B)	0	1	00H	Even Number
35H (00110101B)	1	0	01H	Odd Number

Q2: Positive / Negative Check (2050H) → 2051H (00H=Pos, 01H=Neg)

```
LDA 2050H RAL ; Rotate MSB into Carry JC IS_NEG
MVI A,00H ; Positive (MSB=0) JMP STORE_SGN IS_NEG: MVI
A,01H ; Negative (MSB=1) STORE_SGN: STA 2051H HLT
```

Input (2050H)	MSB	CY Flag	Stored (2051H)	Remarks
45H (01000101B)	0	0	00H	Positive Value
82H (10000010B)	1	1	01H	Negative Value

INTERMEDIATE PRACTICE PROBLEMS (Q1 & Q2)

Q1: Largest of Three Numbers (2050H..2052H) → 2053H

```
LDA 2050H MOV B,A ; B = Largest LDA 2051H CMP B
JC SKP1 MOV B,A ; Update Largest SKP1: LDA 2052H CMP B
JC SKP2 MOV B,A SKP2: MOV A,B STA 2053H HLT
```

Addr	Data	Largest (B)	CMP Flag	Action
2050H	25H	25H	--	Initial Largest
2051H	7AH	7AH	CY=0 (7AH>25H)	Updated to 7AH
2052H	42H	7AH	CY=1 (42H<7AH)	Retained 7AH
Output	2053H ← 7AH (Largest)			

Q2: Smallest of Three Numbers (2050H..2052H) → 2054H

```
LDA 2050H MOV B,A ; B = Smallest LDA 2051H CMP
B JNC SKP1 MOV B,A ; Update Smallest SKP1: LDA 2052H CMP
B JNC SKP2 MOV B,A SKP2: MOV A,B STA 2054H HLT
```

Addr	Data	Smallest (B)	CMP Flag	Action
2050H	4CH	4CH	--	Initial Smallest
2051H	18H	18H	CY=1 (18H<4CH)	Updated to 18H
2052H	32H	18H	CY=0 (32H>18H)	Retained 18H
Output	2054H ← 18H (Smallest)			

ADVANCED PRACTICE PROBLEMS (Q1 & Q2)

Q1: Compare Two 5-Element Arrays & Count Matches → 2070H

```
LXI H,2050H ; Array 1 LXI D,2060H ; Array 2 MVI
C,05H ; Size = 5 MVI B,00H ; Matches CMP_LP: LDAX D CMP
M JNZ NO_MATCH INR B NO_MATCH: INX H INX D DCR C JNZ
CMP_LP MOV A,B STA 2070H HLT
```

Idx	Arr1 (HL)	Arr2 (DE)	Status	Match (B)
1	10H	10H	Equal	01H
2	20H	25H	Not Equal	01H
3	30H	30H	Equal	02H
4	40H	48H	Not Equal	02H
5	50H	50H	Equal	03H
Res	2070H ← 03H (Total Matches)			

Q2: In-Range Validation [2051H=Min, 2052H=Max] → 2053H

```
LDA 2050H ; Val X MOV B,A LDA 2051H ; Min CMP B
; Min - X JNC CHK_EQ1 ; If Min > X, out JMP CHK_MAX
CHK_EQ1: JZ CHK_MAX ; If Min == X, ok JMP OUT_OF_RG
CHK_MAX: LDA 2052H ; Max CMP B ; Max - X JC OUT_OF_RG ;
Max < X -> out IN_RG: MVI A,01H JMP ST_RG OUT_OF_RG: MVI
A,00H ST_RG: STA 2053H HLT
```

Val (2050H)	Min (2051H)	Max (2052H)	Status (2053H)
25H	10H	30H	01H (In Range)
05H	10H	30H	00H (Below Min)
45H	10H	30H	00H (Above Max)

EXPERIMENT NO. 3 • ARITHMETIC APPLICATIONS

16-Bit Subtraction, Multi-Byte Addition, Repeated Arithmetic, and Division

BASIC PRACTICE PROBLEMS (Q1 & Q2)

Q1: Subtract Two 16-Bit Numbers (2050H–51H – 2052H–53H) → 2054H–55H

```
LHLD 2050H ; Minuend in HL XCHG ; Move to DE
LHLD 2052H ; Subtrahend in HL MOV A,E SUB L ; Lower byte
MOV L,A MOV A,D SBB H ; Upper byte with borrow MOV H,A
SHLD 2054H ; Store difference HLT
```

Byte	Minuend (DE)	Subtrahend (HL)	Diff	Result Loc
Lower	67H (E)	34H (L)	33H	2054H ← 33H
Upper	45H (D)	12H (H)	33H	2055H ← 33H
Total	4567H – 1234H = 3333H			

Q2: Add Three 8-Bit Numbers (2050H..2052H) → Store at 2053H

```
LDA 2050H MOV B,A LDA 2051H ADD B MOV B,A LDA
2052H ADD B STA 2053H HLT
```

Inst	A	B	CY	Memory Update
LDA 2050H	15H	–	0	1st byte loaded
ADD B	3AH	15H	0	15H + 25H = 3AH
ADD B	4EH	3AH	0	3AH + 14H = 4EH
STA 2053H	4EH	3AH	0	2053H ← 4EH

INTERMEDIATE PRACTICE PROBLEMS (Q1 & Q2)

Q1: Multiply 8-Bit Number (2050H) by 10 (0AH) → 2051H

```
LDA 2050H MOV B,A ; Multiplicand MVI C,0AH ;
Multiplier = 10 XRA A ; Product = 0 M10_LP: ADD B DCR C
JNZ M10_LP STA 2051H HLT
```

Iter	Multiplicand (B)	Product (A)	Counter (C)
1	06H	06H	09H
2	06H	0CH	08H
...	06H	...	...
10	06H	3CH (60 dec)	00H (Done)
Store	2051H ← 3CH		

Q2: Divide 8-Bit Number (2050H) by 2 → Store Quotient at 2051H

```
LDA 2050H MOV B,A ; Dividend MVI C,00H ;
Quotient = 0 MVI D,02H ; Divisor = 2 D2_LP: MOV A,B CMP
D JC D2_END SUB D MOV B,A INR C JMP D2_LP D2_END:MOV A,C
STA 2051H HLT
```

Dividend (B)	Divisor (D)	Quotient (C)	Remarks
0FH (15)	02H	00H	Start
0DH (13)	02H	01H	15 – 2 = 13
...	02H	...	...
01H (1)	02H	07H	1 < 2 (Exit)
Result	2051H ← 07H (Quotient)		

ADVANCED PRACTICE PROBLEMS (Q1 & Q2)

Q1: Multi-Byte Multiplication (16-Bit Precision) → 2052H–53H

```
LDA 2050H ; Multiplier MOV C,A LDA 2051H ;
Multiplicand MOV E,A MVI D,00H ; DE = Multiplicand LXI
H,0000H ; Clear Product MOV A,C ORA A JZ ST_PROD M16_LP:
DAD D ; HL = HL + DE DCR C JNZ M16_LP ST_PROD: SHLD
2052H HLT
```

Iter	Multiplier (C)	DE	HL (Running Product)
Init	12H (18)	0015H (21)	0000H
1	11H	0015H	0015H
2	10H	0015H	002AH
18	00H	0015H	017AH (378 dec)
Store	2052H ← 7AH, 2053H ← 01H		

Q2: 8-Bit Division with Quotient (2052H) & Remainder (2053H)

```
LDA 2050H ; Dividend MOV B,A LDA 2051H ;
Divisor MOV C,A MVI D,00H ; Quotient = 0 DIV_LP: MOV A,B
CMP C JC DIV_FIN SUB C MOV B,A INR D JMP DIV_LP DIV_FIN:
MOV A,D STA 2052H ; Quotient MOV A,B STA 2053H ;
Remainder HLT
```

Step	Dividend (B)	Divisor (C)	Quotient (D)
Start	17H (23)	05H (5)	00H
Sub 1	12H (18)	05H	01H
Sub 2	0DH (13)	05H	02H
Sub 3	08H (8)	05H	03H
Sub 4	03H (3)	05H	04H
Exit	03H < 05H → Quot: 04H, Rem: 03H		

EXPERIMENT NO. 4 • DATA PROCESSING & ARRAY OPERATIONS

Array Averages, Even Counts, Linear Search, Sign Tally, Block Copy, and Inversion

BASIC PRACTICE PROBLEMS (Q1 & Q2)

Q1: Calculate Average of Five 8-Bit Numbers → Store at 2060H

```
LXI H,2050H MVI B,05H ; N = 5 XRA A ; Sum = 0
AV_LP: ADD M INX H DCR B JNZ AV_LP MVI B,00H ; Quotient
MVI C,05H ; Divisor = 5 D_AV: CMP C JC ST_AV SUB C INR B
JMP D_AV ST_AV: MOV A,B STA 2060H HLT
```

Array Elements (2050H..54H)	Sum	Divisor	Average (2060H)
10H, 14H, 0AH, 0FH, 15H	4CH (76)	5	0FH (15 dec)

Q2: Count Number of Even Elements in Array (N=5) → 2061H

```
LXI H,2050H MVI B,05H ; Count = 5 MVI C,00H ;
Even count EV_LP: MOV A,M ANI 01H ; Test LSB JNZ
ODD_SKIP INR C ODD_SKIP: INX H DCR B JNZ EV_LP MOV A,C
STA 2061H HLT
```

Element	ANI 01H	Z Flag	Even Count (C)
12H (18)	00H	1	01H
17H (23)	01H	0	01H
24H (36)	00H	1	02H
31H (49)	01H	0	02H
40H (64)	00H	1	03H at 2061H

INTERMEDIATE PRACTICE PROBLEMS (Q1 & Q2)

Q1: Linear Search for Key (204FH) in Array → 2060H (01H/00H)

```
LDA 204FH ; Search Key MOV B,A LXI H,2050H ;
Array start MVI C,05H ; 5 elements SRCH_LP: MOV A,M CMP
B JZ KEY_FND INX H DCR C JNZ SRCH_LP MVI A,00H ; Not
found JMP ST_SRCH KEY_FND: MVI A,01H ; Found ST_SRCH:
STA 2060H HLT
```

Idx	Element	Key (B)	CMP Flag	Action
1	15H	32H	CY=1	Next
2	28H	32H	CY=1	Next
3	32H	32H	Z=1	Match found!
Res	2060H ← 01H (Key Found)			

Q2: Count Positive & Negative Numbers → 2060H & 2061H

```
LXI H,2050H MVI B,05H ; Counter = 5 MVI C,00H ;
Pos count MVI D,00H ; Neg count SGN_LP: MOV A,M RAL ;
MSB to Carry JC IS_NEGATIVE INR C JMP SGN_CONT
IS_NEGATIVE: INR D SGN_CONT: INX H DCR B JNZ SGN_LP MOV
A,C STA 2060H ; Pos count MOV A,D STA 2061H ; Neg count
HLT
```

Data	RAL (CY)	Pos (C)	Neg (D)
12H (+)	0	01H	00H
85H (-)	1	01H	01H
30H (+)	0	02H	01H
FEH (-)	1	02H	02H
7AH (+)	0	03H	02H
Outputs	2060H=03H, 2061H=02H		

ADVANCED PRACTICE PROBLEMS (Q1 & Q2)

Q1: Copy 5-Element Array from 2050H to 2060H

```
LXI H,2050H ; Source LXI D,2060H ; Dest MVI
C,05H ; Length CPY_LP: MOV A,M STAX D INX H INX D DCR C
JNZ CPY_LP HLT
```

Iter	Source (HL)	Data	Dest (DE)	Counter (C)
1	2050H	11H	2060H	04H
2	2051H	22H	2061H	03H
3	2052H	33H	2062H	02H
4	2053H	44H	2063H	01H
5	2054H	55H	2064H	00H (Done)

Q2: In-Place Reverse of 5-Element Array (2050H..2054H)

```
LXI H,2050H ; Left ptr LXI D,2054H ; Right ptr
MVI B,02H ; Swaps = 5/2 = 2 REV_LP: LDAX D MOV C,A ;
Cache right MOV A,M ; Read left STAX D ; Store left at
right MOV M,C ; Store right at left INX H DCX D DCR B
JNZ REV_LP HLT
```

Iter	Left (HL)	Right (DE)	Swapped Elements	State
Init	2050H	2054H	–	[1, 2, 3, 4, 5]
1	2050H	2054H	11H ↔ 55H	[5, 2, 3, 4, 1]
2	2051H	2053H	22H ↔ 44H	[5, 4, 3, 2, 1]
End	Array Fully Inverted In-Place			

EXPERIMENT NO. 5 • LOGICAL & BIT MANIPULATION PROGRAMS

Bit Inversion, Circular Rotations, Flag Tests, Nibble Swapping, and Complements

BASIC PRACTICE PROBLEMS (Q1 & Q2)

Q1: Complement 8-Bit Number (2050H) → 2051H

```
LDA 2050H CMA ; Invert all bits (1's comp) STA
2051H HLT
```

Input (2050H)	Binary Pattern	CMA Output	Stored (2051H)
55H	01010101B	10101010B	AAH
0FH	00001111B	11110000B	F0H

Q2: Rotate 8-Bit Number Left by 1 Bit (2050H) → 2051H

```
LDA 2050H RLC ; Rotate left circular (D7 to D0
& CY) STA 2051H HLT
```

Input (2050H)	Before RLC	After RLC	CY	Stored (2051H)
81H	10000001B	00000011B	1	03H
40H	01000000B	10000000B	0	80H

INTERMEDIATE PRACTICE PROBLEMS (Q1 & Q2)

Q1: Rotate 8-Bit Number Right through Carry → 2051H

```
LDA 2050H RAR ; Rotate right through Carry STA
2051H HLT
```

Input	Initial CY	After RAR	New CY	Stored (2051H)
05H (00000101)	1	10000010B	1	82H
04H (00000100)	0	00000010B	0	02H

Q2: Check Bit-3 Status (2050H) → 2051H (01H=Set, 00H=Reset)

```
LDA 2050H ANI 08H ; Mask Bit 3 (00001000B) JZ
BIT_RESET MVI A,01H ; Bit is 1 JMP ST_B3 BIT_RESET: MVI
A,00H ; Bit is 0 ST_B3: STA 2051H HLT
```

Input (2050H)	Mask (08H)	Z Flag	Stored (2051H)	Remarks
2BH (00101011B)	08H	0	01H	Bit 3 is Set
23H (00100011B)	00H	1	00H	Bit 3 is Reset

ADVANCED PRACTICE PROBLEMS (Q1 & Q2)

Q1: Swap Upper & Lower Nibbles (2050H) → 2051H

```
LDA 2050H RRC RRC RRC RRC ; 4 rotations swap
nibbles STA 2051H HLT
```

Input (2050H)	Upper Nibble	Lower Nibble	Swapped (2051H)
A5H	AH	5H	5AH
3FH	3H	FH	F3H

Q2: Generate 1's Comp (2051H) & 2's Comp (2052H)

```
LDA 2050H CMA ; 1's complement STA 2051H INR A
; 2's complement = 1's comp + 1 STA 2052H HLT
```

Input (2050H)	1's Comp (2051H)	2's Comp (2052H)	Verification
06H (00000110B)	F9H (11111001B)	FAH (11111010B)	06H + FAH = 100H
25H (00100101B)	DAH (11011010B)	DBH (11011011B)	25H + DBH = 100H

EXPERIMENT NO. 6 • LOOPING, DELAY & COUNTER PROGRAMS

Multi-Register Delays, Hex Counters, 1-Second Timers, LED Blinkers, and Configurable Loops

BASIC PRACTICE PROBLEMS (Q1 & Q2)

Q1: Nested-Loop Software Delay Routine

```
MVI B,FFH ; Outer count OUT_LP: MVI C,FFH ;  
Inner count IN_LP: DCR C JNZ IN_LP DCR B JNZ OUT_LP HLT
```

Loop Tier	Counter	T-States/Iter	Total T-States (approx)
Inner	C = 255	14 T-states	$255 \times 14 \approx 3,570$
Outer	B = 255	$3,570 + 14$	$255 \times 3,584 \approx 914,000$ (~0.45s at 2MHz)

Q2: Hex Counter from 00H to FFH → Store Count at 2050H

```
MVI A,00H CNT_LP: STA 2050H INR A CPI 00H ;  
Rollover check JNZ CNT_LP HLT
```

Iter	Accumulator (A)	2050H Updated	Z Flag
1	00H	$2050H \leftarrow 00H$	0
2	01H	$2050H \leftarrow 01H$	0
256	FFH	$2050H \leftarrow FFH$	0
257	00H	Rollover (CPI 00H)	1 (Halt)

INTERMEDIATE PRACTICE PROBLEMS (Q1 & Q2)

Q1: 1-Second Delay Routine via 16-Bit Register Pair Loop

```
LXI D,3E80H ; Count = 16,000 dec D1S_LP: DCX D  
; 6 T-states MOV A,E; 4 T-states ORA D ; 4 T-states JNZ  
D1S_LP ; 10 T-states HLT
```

Instruction	T-States	Iterations	Total Cycles
DCX D + MOV + ORA + JNZ	24	16,000	384,000 T-states
Pair with outer loop ×5	--	--	~2,000,000 T-states = 1.00s at 2MHz

Q2: Count Iterations Required for Given Delay Count → 2051H

```
LDA 2050H ; Delay Count MOV B,A MVI C,00H ;  
Iteration counter IT_LP: INR C ; Tally iteration DCR B  
JNZ IT_LP MOV A,C STA 2051H HLT
```

Initial Count (2050H)	Reg B	Reg C (Tally)	Stored (2051H)
05H	$05H \rightarrow 00H$	$00H \rightarrow 05H$	05H
64H (100)	$64H \rightarrow 00H$	$00H \rightarrow 64H$	64H (100 dec)

ADVANCED PRACTICE PROBLEMS (Q1 & Q2)

Q1: Blink LED Continuously on Port 01H with Delay

```
LXI SP,30FFH BLINK: MVI A,FFH ; LED ON OUT 01H  
CALL DELAY MVI A,00H ; LED OFF OUT 01H CALL DELAY JMP  
BLINK DELAY: MVI B,20H D_IN: DCR B JNZ D_IN RET
```

Phase	Acc	Port 01H	Subroutine
State 1	FFH	FFH (ON)	CALL DELAY executed
State 2	00H	00H (OFF)	CALL DELAY executed
Cycle	Continuous square-wave LED toggling		

Q2: Configurable Delay via Memory Parameter at 2050H

```
LDA 2050H ; Multiplier MOV D,A VAR_OUT: MVI  
E,FFH VAR_IN: DCR E JNZ VAR_IN DCR D JNZ VAR_OUT HLT
```

Param (2050H)	Outer (D)	Inner (E)	Effective Duration
01H	1	255	Fast (~2.5 ms)
40H (64)	64	255	Medium (~160 ms)
FFH (255)	255	255	Extended (~650 ms)

EXPERIMENT NO. 7 • STACK & SUBROUTINE PROGRAMMING

Stack Transfers, Subroutine Arithmetic, 3-Pair Reversals, and Nested Hierarchy

BASIC PRACTICE PROBLEMS (Q1 & Q2)

Q1: Transfer Contents of HL to DE Using Stack

```
LXI SP,30FFH ; Init Stack Pointer
LXI H,1234H ; Test data in HL
PUSH H ; Store HL on stack
POP D ; Pop into DE
HLT
```

Inst	SP	TOS Value	HL	DE
LXI SP	30FFH	--	--	--
LXI H	30FFH	--	1234H	--
PUSH H	30FDH	1234H	1234H	--
POP D	30FFH	--	1234H	1234H

Q2: Subtract Two 8-Bit Numbers Using Subroutine → 2052H

```
LXI SP,30FFH LDA 2050H MOV B,A LDA 2051H CALL
SUBTRACT STA 2052H HLT SUBTRACT: MOV C,A MOV A,B SUB C
RET
```

Stage	A	B	C	SP	Memory
Main Load	15H	40H	--	30FFH	--
CALL	15H	40H	--	30FDH	Ret pushed
Sub Exec	2BH	40H	15H	30FDH	40H - 15H = 2BH
RET	2BH	40H	15H	30FFH	2052H ← 2BH

INTERMEDIATE PRACTICE PROBLEMS (Q1 & Q2)

Q1: Multiply Two 8-Bit Numbers via Subroutine → 2052H

```
LXI SP,30FFH LDA 2050H MOV B,A LDA 2051H MOV
C,A CALL MULTIPLY STA 2052H HLT MULTIPLY: XRA A MULT_LP:
ADD B DCR C JNZ MULT_LP RET
```

Inputs	Subroutine Iterations	Accumulator	Stored (2052H)
B=05H, C=04H	4 additions of 05H	14H (20 dec)	2052H ← 14H
B=08H, C=06H	6 additions of 08H	30H (48 dec)	2052H ← 30H

Q2: Reverse Contents of 3 Register Pairs (BC, DE, HL) via Stack

```
LXI SP,30FFH LXI B,1111H LXI D,2222H LXI
H,3333H PUSH B ; Pushed 1st
PUSH D ; Pushed 2nd
PUSH H ; Pushed 3rd (TOS)
POP B ; Pop 1st -> BC gets 3333H
POP D ; Pop 2nd -> DE gets 2222H
POP H ; Pop 3rd -> HL gets 1111H
HLT
```

Operation	SP	BC	DE	HL
Start	30FFH	1111H	2222H	3333H
After 3 PUSHes	30F9H	--	--	--
POP B	30FBH	3333H	--	--
POP D	30FDH	3333H	2222H	--
POP H	30FFH	3333H	2222H	1111H

ADVANCED PRACTICE PROBLEMS (Q1 & Q2)

Q1: Evaluate Expression with Multiple Subroutines → 2054H

```
LXI SP,30FFH LDA 2050H MOV B,A LDA 2051H CALL
SUB_ADD ; (A + B)
MOV D,A LDA 2052H MOV B,A LDA 2053H
CALL SUB_ADD ; (C + D)
MOV E,A CALL SUB_DIFF ; D - E
STA 2054H HLT SUB_ADD: ADD B RET
SUB_DIFF: MOV A,D SUB E RET
```

Subroutine	Operands	Result	Dest Reg
SUB_ADD (1)	A=20H, B=15H	35H	Reg D = 35H
SUB_ADD (2)	C=10H, D=05H	15H	Reg E = 15H
SUB_DIFF	D=35H, E=15H	20H	2054H ← 20H

Q2: Nested Subroutines: MAIN → SUB_A → SUB_B

```
LXI SP,30FFH LDA 2050H ; Start = 02H
CALL SUB_A STA 2051H HLT SUB_A: INR A ; A = 03H
CALL SUB_B RLC ; A = 10H
RET SUB_B: ADI 05H ; A = 08H
RET
```

Point	SP	Acc	TOS Contains
In SUB_A	30FDH	03H	Main Return
In SUB_B	30FBH	08H	SUB_A Return
After B RET	30FDH	08H	Main Return
After RLC	30FFH	10H	2051H ← 10H

EXPERIMENT NO. 8 • CODE CONVERSION & BCD ARITHMETIC

Excess-3, BCD-to-Binary, ASCII Encoding, 10's Complement BCD Subtraction, and Gray Arrays

BASIC PRACTICE PROBLEMS (Q1 & Q2)

Q1: Convert Binary to Excess-3 Code (2050H) → 2051H

```
LDA 2050H ADI 03H ; Excess-3 = Binary + 03H STA 2051H HLT
```

Binary Input (2050H)	Decimal	+ 03H	Excess-3 (2051H)
05H	5	05H + 03H	08H
09H	9	09H + 03H	0CH

Q2: Convert Single-Digit BCD to Binary (2050H) → 2051H

```
LDA 2050H ANI 0FH ; Isolate valid BCD digit STA 2051H HLT
```

BCD Input (2050H)	Mask (0FH)	Binary Output (2051H)
07H	07H AND 0FH	07H
09H	09H AND 0FH	09H

INTERMEDIATE PRACTICE PROBLEMS (Q1 & Q2)

Q1: Convert Hex/Binary Nibble to ASCII (2050H) → 2051H

```
LDA 2050H ANI 0FH ; Lower nibble CPI 0AH ; Compare with 10 JC IS_NUM ADI 07H ; Add letter offset ('A'-'F') IS_NUM: ADI 30H ; Add '0' base STA 2051H HLT
```

Nibble	CPI 0AH	Adjustment	ASCII Output (2051H)
05H	CY=1	+ 30H	35H ('5')
0BH	CY=0	+ 07H + 30H	42H ('B')

Q2: BCD Subtraction via 10's Complement (2050H - 2051H) → 2052H

```
MVI A,99H ; 9's complement base LDA 2051H ; Subtrahend MOV B,A MOV A,MVI_99: MVI A,99H SUB B ; 9's complement INR A ; 10's complement MOV B,A LDA 2050H ; Minuend ADD B DAA ; Decimal adjust STA 2052H HLT
```

Minuend	Subtrahend	10's Comp	DAA Result
52H (BCD 52)	25H (BCD 25)	99H-25H+1 = 75H	52H+75H=C7H → 27H (BCD 27)

ADVANCED PRACTICE PROBLEMS (Q1 & Q2)

Q1: Two-Digit Packed BCD to Binary (2050H) → 2051H

```
LDA 2050H MOV B,A ANI 0FH MOV C,A ; Units MOV A,B ANI F0H RRC RRC RRC RRC ; Tens digit MOV D,A XRA A MVI E,0AH ; 10 dec MOV H,D ORA H JZ ADD_U TEN_LP: ADD E ; Tens * 10 DCR D JNZ TEN_LP ADD_U: ADD C ; Add units STA 2051H HLT
```

Packed BCD	Tens × 10	Units	Binary (2051H)
25H (BCD 25)	2 × 10 = 20 (14H)	05H	14H + 05H = 19H (25 dec)
68H (BCD 68)	6 × 10 = 60 (3CH)	08H	3CH + 08H = 44H (68 dec)

Q2: Convert Binary Array to Gray Code (2050H..54H → 2060H..64H)

```
LXI H,2050H ; Source LXI D,2060H ; Destination MVI C,05H ; Count = 5 GRAY_LP: MOV A,M MOV B,A RRC ; Shift right 1 XRA B ; Gray = B ^ (B >> 1) STAX D INX H INX D DCR C JNZ GRAY_LP HLT
```

Binary In	Binary Pattern	RRC	Gray Out
0DH	00001101B	10000110B	0BH
05H	00000101B	10000010B	07H
08H	00001000B	00000100B	0CH

EXPERIMENT NO. 9 • ADVANCED ARITHMETIC & MATHEMATICAL PROGRAMS

Powers, LCM Algorithms, Fibonacci Series Generation, Composite Products, and Primality

BASIC PRACTICE PROBLEMS (Q1 & Q2)

Q1: Square of 8-Bit Number (2050H) via Repeated Addition → 2051H

```
LDA 2050H MOV B,A ; N MOV C,A ; Counter N XRA A
; Square = 0 SQ_LP: ADD B DCR C JNZ SQ_LP STA 2051H HLT
```

Input N (2050H)	Calculation	Product (Hex)	Stored (2051H)
05H	5 × 5 = 25	19H	19H
08H	8 × 8 = 64	40H	40H

Q2: Cube of 8-Bit Number (2050H) → 2052H

```
LDA 2050H MOV B,A MOV C,A XRA A S_LP: ADD B ; N
* N DCR C JNZ S_LP MOV D,A ; D = N^2 LDA 2050H MOV C,A ;
Multiplier N XRA A C_LP: ADD D ; N^2 * N DCR C JNZ C_LP
STA 2052H HLT
```

N	Square (D)	Cube Formula	Stored (2052H)
03H	09H	9 × 3 = 27	1BH
04H	10H (16)	16 × 4 = 64	40H

INTERMEDIATE PRACTICE PROBLEMS (Q1 & Q2)

Q1: Least Common Multiple (LCM) of Two Numbers → 2052H

```
LDA 2050H MOV B,A ; N1 LDA 2051H MOV C,A ; N2
MOV D,B ; Mult 1 MOV E,C ; Mult 2 LCM_LP: MOV A,D CMP E
JZ LCM_DONE JC INC_M1 MOV A,E ADD C MOV E,A JMP LCM_LP
INC_M1: MOV A,D ADD B MOV D,A JMP LCM_LP LCM_DONE: STA
2052H HLT
```

Iter	Mult 1 (D)	Mult 2 (E)	CMP Flag	Action
0	04H	06H	CY=1	D = 4 + 4 = 8
1	08H	06H	CY=0	E = 6 + 6 = 12
2	08H	0CH	CY=1	D = 8 + 4 = 12
3	0CH	0CH	Z=1	LCM = 0CH (12 dec)

Q2: Generate First N Terms of Fibonacci Series (N at 2050H)

```
LDA 2050H ; N terms MOV D,A LXI H,2060H ;
Output start MVI B,00H ; Term 1 = 0 MVI C,01H ; Term 2 =
1 MOV M,B INX H DCR D JZ FIB_DONE MOV M,C INX H DCR D JZ
FIB_DONE FIB_LP: MOV A,B ADD C ; Next = T1 + T2 MOV M,A
MOV B,C MOV C,A INX H DCR D JNZ FIB_LP FIB_DONE: HLT
```

Term #	Address	Value (Hex)	Decimal Value
1	2060H	00H	0
2	2061H	01H	1
3	2062H	01H	1
4	2063H	02H	2
5	2064H	03H	3
6	2065H	05H	5

ADVANCED PRACTICE PROBLEMS (Q1 & Q2)

Q1: Evaluate (A × B) + (C × D) with 16-Bit Product Sum

```
LDA 2050H MOV B,A LDA 2051H MOV C,A LXI H,0000H
; P1 = 0 M1: MOV A,C ORA A JZ M1_END MOV E,B MVI D,00H
DAD D DCR C JMP M1 M1_END: SHLD 2054H ; Cache P1 LDA
2052H MOV B,A LDA 2053H MOV C,A LXI H,0000H ; P2 = 0 M2:
MOV A,C ORA A JZ M2_END MOV E,B MVI D,00H DAD D DCR C
JMP M2 M2_END: XCHG ; DE = P2 LHLD 2054H ; HL = P1 DAD D
; HL = P1 + P2 SHLD 2054H HLT
```

Expression Term	Operands	Product	Running Total
A × B	05H × 04H	0014H (20)	0014H
C × D	06H × 03H	0012H (18)	–
Composite	0014H + 0012H = 0026H (38 dec) at 2054H–55H		

Q2: Primality Test (2050H) → 2051H (01H=Prime, 00H=Composite)

```
LDA 2050H ; N CPI 02H JC NOT_PRIME JZ IS_PRIME
MOV B,A ; N MVI C,02H ; Divisor = 2 PR_LP: MOV A,C CMP B
JZ IS_PRIME MOV D,B MOD_LP:MOV A,D CMP C JC NXT_DIV JZ
NOT_PRIME SUB C MOV D,A JMP MOD_LP NXT_DIV: INR C JMP
PR_LP IS_PRIME: MVI A,01H JMP ST_PR NOT_PRIME: MVI A,00H
ST_PR: STA 2051H HLT
```

N (2050H)	Divisors Tested	Remainder	Status (2051H)
07H	2, 3, 4, 5, 6	Non-zero for all	01H (Prime)
09H	2, 3	9 mod 3 = 0	00H (Composite)

EXPERIMENT NO. 10 • MEMORY & INPUT/OUTPUT PROGRAMMING

Block Transfers, Memory Exchange, Port Relays, Block Initialization, and Filtered Copies

BASIC PRACTICE PROBLEMS (Q1 & Q2)

Q1: Copy 20 Bytes from 2050H to 2070H

```
LXI H,2050H ; Source
LXI D,2070H ; Destination
MVI C,14H ; 20 dec = 14H
CP20: MOV A,M
STAX D
INX H
INX D
DCR C
JNZ CP20
HLT
```

Iter	Source (HL)	Data Read	Dest (DE)	Remaining (C)
1	2050H	AAH	2070H	13H
2	2051H	BBH	2071H	12H
...	...	...	...	...
20	2063H	FFH	2083H	00H (Terminated)

Q2: Exchange Contents of 2050H & 2051H

```
LDA 2050H
MOV B,A ; Cache 2050H
LDA 2051H
STA 2050H ; 2050H = Old 2051H
MOV A,B
STA 2051H ; 2051H = Old 2050H
HLT
```

Step	Addr 2050H	Addr 2051H	Reg B	Remarks
Init	12H	34H	--	Initial Values
Step 1	12H	34H	12H	2050H cached
Step 2	34H	34H	12H	2050H updated
Step 3	34H	12H	12H	Swapped

INTERMEDIATE PRACTICE PROBLEMS (Q1 & Q2)

Q1: Relay Data from Input Port 03H to Output Port 04H

```
IN 03H ; Read Port 03H
OUT 04H ; Send to Port 04H
HLT
```

Inst	Acc	Input Port 03H	Output Port 04H
IN 03H	5AH	5AH	--
OUT 04H	5AH	5AH	5AH (Transmitted)

Q2: Clear 16 Consecutive Memory Locations (2050H..205FH)

```
LXI H,2050H
MVI C,10H ; 16 dec = 10H
XRA A ; A = 00H
CLR16: MOV M,A
INX H
DCR C
JNZ CLR16
HLT
```

Iter	Address (HL)	Data Written	Counter (C)
1	2050H	00H	0FH
2	2051H	00H	0EH
...	...	...	...
16	205FH	00H	00H (Block Cleared)

ADVANCED PRACTICE PROBLEMS (Q1 & Q2)

Q1: Copy Only Even-Valued Bytes from 2050H Block to 2070H Block

```
LXI H,2050H ; Source
LXI D,2070H ; Dest
MVI B,0AH ; 10 bytes
EV_BLK: MOV A,M
ANI 01H ; Test LSB
JNZ SKP_ODD
MOV A,M
STAX D
INX D
SKP_ODD: INX H
DCR B
JNZ EV_BLK
HLT
```

Src Addr	Data	Even/Odd	Copied to DE?
2050H	14H	Even	2070H ← 14H
2051H	23H	Odd	Skipped
2052H	48H	Even	2071H ← 48H
2053H	19H	Odd	Skipped

Q2: Count Non-Zero Bytes in Block (2050H..5FH) → 2080H

```
LXI H,2050H
MVI C,10H ; 16 bytes
MVI B,00H ; Non-zero tally
NZ_LP: MOV A,M
ORA A ; Test if zero
JZ IS_ZR
INR B
IS_ZR: INX H
DCR C
JNZ NZ_LP
MOV A,B
STA 2080H
HLT
```

Block Content Sample	Non-Zero Count	Stored (2080H)
10H, 00H, 05H, 00H, 00H, 32H...	7 non-zero elements	2080H ← 07H

EXPERIMENT NO. 11 • APPLICATION-ORIENTED PROGRAMMING

Attendance Counters, Security Access Systems, Tank Monitors, Traffic Signals, and ATM Checks

BASIC PRACTICE PROBLEMS (Q1 & Q2)

Q1: Electronic Attendance System (Count 01H in 10 Bytes) → 2060H

```
LXI H,2050H ; Attendance log MVI B,0AH ; 10
students MVI C,00H ; Present tally ATT_LP: MOV A,M CPI
01H ; 01H = Present JNZ ABSENT INR C ABSENT: INX H DCR B
JNZ ATT_LP MOV A,C STA 2060H HLT
```

Array (2050H..59H)	Present (01H)	Absent (00H)	Total Present (2060H)
01, 01, 00, 01, 01, 01, 00, 01, 01, 00	7	3	2060H ← 07H

Q2: Security Access Verifier (Key: 5AH at 2050H) → 2051H

```
LDA 2050H ; Entered ID CPI 5AH ; Valid Key JZ
ACCESS_OK MVI A,00H ; Access Denied JMP ST_ACC
ACCESS_OK: MVI A,01H ; Access Granted ST_ACC: STA 2051H
HLT
```

Entered ID (2050H)	CPI 5AH	Status (2051H)	Meaning
5AH	Z = 1	01H	Access Granted
42H	Z = 0	00H	Access Denied

INTERMEDIATE PRACTICE PROBLEMS (Q1 & Q2)

Q1: Water Tank Level Monitor (2050H) → 2051H (01H/02H/03H)

```
LDA 2050H ; Sensor level CPI 20H ; Low
threshold JC LVL_LOW CPI 61H ; High threshold JNC
LVL_HIGH MVI A,02H ; Status = NORMAL JMP ST_TANK
LVL_LOW: MVI A,01H ; Status = LOW JMP ST_TANK LVL_HIGH:
MVI A,03H ; Status = HIGH ST_TANK: STA 2051H HLT
```

Reading (2050H)	Range	Status (2051H)	Indicator
15H	< 20H	01H	LOW
45H	20H to 60H	02H	NORMAL
72H	> 60H	03H	HIGH

Q2: Traffic Signal Controller Sequence on Port 01H

```
LXI SP,30FFH TF_LP: MVI A,01H ; RED Light OUT
01H CALL DELAY MVI A,02H ; YELLOW Light OUT 01H CALL
DELAY MVI A,04H ; GREEN Light OUT 01H CALL DELAY JMP
TF_LP DELAY: MVI B,10H D_TF: DCR B JNZ D_TF RET
```

Phase	Port 01H Output	Active Signal	Delay Active
1	01H (00000001B)	RED Light	Yes
2	02H (00000010B)	YELLOW Light	Yes
3	04H (00000100B)	GREEN Light	Yes

ADVANCED PRACTICE PROBLEMS (Q1 & Q2)

Q1: ATM Balance Check & Cash Dispense System

```
LDA 2050H ; Balance MOV B,A LDA 2051H ;
Withdrawal Request CMP B ; Request - Balance JC VALID_TX
; If Request < Balance (CY=1) JZ VALID_TX ; If Request
== Balance ; Overdraft / Insufficient Funds MVI A,EEH ;
Error code STA 2052H HLT VALID_TX: MOV C,A ; C = Request
MOV A,B ; A = Balance SUB C ; Remaining Balance STA
2050H ; Update account MVI A,01H ; Success code STA
2052H HLT
```

Balance	Request	Updated Balance	Status (2052H)
50H (\$80)	20H (\$32)	30H (\$48)	01H (Success)
20H (\$32)	40H (\$64)	20H (Unchanged)	EEH (Denied)

Q2: Multi-Sensor Critical Alarm Trigger (5 Sensors)

```
LXI H,2050H ; Sensors LXI D,2060H ; Limits MVI
B,05H ; 5 channels MVI C,00H ; Breaches SCAN: LDAX D ;
Threshold CMP M ; Threshold - Sensor JNC OK ; Sensor <=
Limit INR C ; Sensor > Limit! OK: INX H INX D DCR B JNZ
SCAN MOV A,C STA 2070H ; Breaches count ORA A JZ
NO_ALARM MVI A,FFH ; Alarm ON OUT 01H HLT NO_ALARM: MVI
A,00H OUT 01H HLT
```

Channel	Sensor (HL)	Limit (DE)	Status
CH1	32H	40H	Normal
CH2	55H	45H	BREACH!
Alarm	Port 01H = FFH (Tripped), 2070H = 01H		

EXPERIMENT NO. 12 • INTEGRATED APPLICATIONS & SYSTEM CHALLENGES

Complete Verification & Dry Run Tables for Student Result Processing & Inventory Replenishment

PROGRAM 1: STUDENT RESULT PROCESSING SYSTEM (PASS MARK ≥ 23H [35 DEC])

Inputs (2050H–2059H): 48H (72), 3CH (60), 21H (33), 55H (85), 2DH (45), 1EH (30), 41H (65), 32H (50), 5AH (90), 24H (36).

Student	Address	Mark (Hex/Dec)	Running Total (LSB)	Highest (B)	Lowest (C)	Pass/Fail Status
S1	2050H	48H (72)	48H	48H	48H	Pass (Pass=1, Fail=0)
S2	2051H	3CH (60)	84H	48H	3CH	Pass (Pass=2, Fail=0)
S3	2052H	21H (33)	A5H	48H	21H	FAIL (Pass=2, Fail=1)
S4	2053H	55H (85)	FAH	55H	21H	Pass (Pass=3, Fail=1)
S5	2054H	2DH (45)	27H (CY=1)	55H	21H	Pass (Pass=4, Fail=1)
S6	2055H	1EH (30)	45H	55H	1EH	FAIL (Pass=4, Fail=2)
S7	2056H	41H (65)	86H	55H	1EH	Pass (Pass=5, Fail=2)
S8	2057H	32H (50)	B8H	55H	1EH	Pass (Pass=6, Fail=2)
S9	2058H	5AH (90)	12H (CY=1)	5AH	1EH	Pass (Pass=7, Fail=2)
S10	2059H	24H (36)	36H	5AH	1EH	Pass (Pass=8, Fail=2)

Report: 2060H = 36H (Sum LSB), 2061H = 5AH (Highest), 2062H = 1EH (Lowest), 2063H = 08H (Pass Count), 2064H = 02H (Fail Count)

PROGRAM 2: INVENTORY MANAGEMENT SYSTEM (THRESHOLD = 0AH [10 DEC])

Inputs (2050H–2059H): 15H, 08H, 0CH, 05H, 12H, 09H, 14H, 07H, 11H, 06H. Status 01H = Replenish, 00H = Stock OK.

Product #	Address	Quantity (Hex/Dec)	Threshold (0AH)	Status Array (2060H–2069H)	Low-Stock Count (2070H)	Remarks
P1	2050H	15H (21)	0AH (10)	2060H ← 00H	00H	Stock Sufficient
P2	2051H	08H (8)	0AH (10)	2061H ← 01H	01H	REPLENISH REQUIRED
P3	2052H	0CH (12)	0AH (10)	2062H ← 00H	01H	Stock Sufficient
P4	2053H	05H (5)	0AH (10)	2063H ← 01H	02H	REPLENISH REQUIRED
P5	2054H	12H (18)	0AH (10)	2064H ← 00H	02H	Stock Sufficient
P6	2055H	09H (9)	0AH (10)	2065H ← 01H	03H	REPLENISH REQUIRED
P7	2056H	14H (20)	0AH (10)	2066H ← 00H	03H	Stock Sufficient
P8	2057H	07H (7)	0AH (10)	2067H ← 01H	04H	REPLENISH REQUIRED
P9	2058H	11H (17)	0AH (10)	2068H ← 00H	04H	Stock Sufficient
P10	2059H	06H (6)	0AH (10)	2069H ← 01H	05H	REPLENISH REQUIRED

Summary Report: 2060H–2069H holds replenishment flags; 2070H = 05H (Total low-stock items requiring reorder)